

Async mechanisms: same lifecycle, different proof obligations

A prototype advisor deck

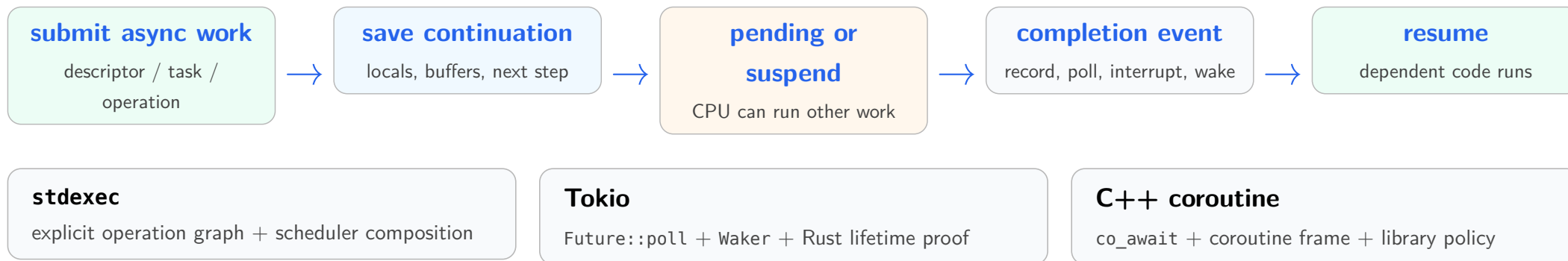
stdexec, Rust Tokio, and C++ coroutine for accelerator-backed async work

Hongtao Zhang · May 2026

The goal is not to teach every framework detail. The goal is to explain what each mechanism makes explicit when a DSA/IAX operation is outstanding.

One async shape behind three mechanisms

Takeaway: all three mechanisms submit work without blocking the CPU, save what must happen next, and resume that continuation after completion.

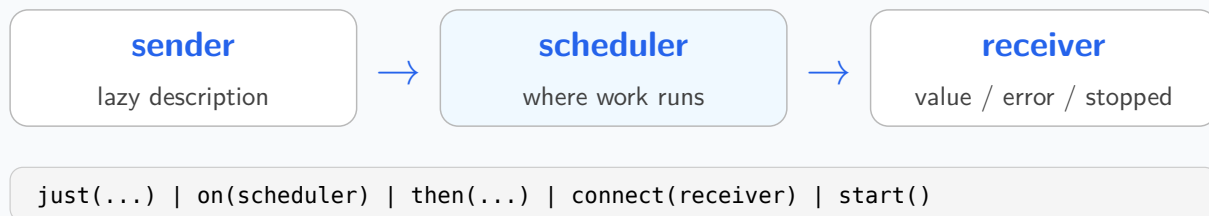


DSA/IAX mapping: hardware has pointers to buffers and a completion record while software wants to continue. The async interface must say who keeps that state valid and who resumes execution.

stdexec: explicit operation graph

Takeaway: stdexec makes the async graph and scheduler boundary visible, which is useful for controlled C++ experiments.

Visual model



How to say it

1. Describe work lazily; starting is a separate step.
2. Completion contract is explicit at the receiver.
3. Scheduler and composition layers can be swapped.
4. This matches the earlier layer-removal experiment style.

Advisor framing: stdexec is good when the research question is “which software layer costs how much?”

Tokio/Rust: polling, wakeups, and lifetime proof

Takeaway: Tokio gives a natural task/waker integration point, but Rust forces us to prove that all state crossing `.await` remains valid.

Runtime view

```
poll(Pin<&mut Future>, Context) -> Pending
```

1. Pending does not block the thread.
2. The future registers a Waker in the context.
3. Completion calls wake; Tokio schedules another poll.
4. State used after `.await` is stored in the task/future.

Lifetime view for hardware

src buffer

must not disappear

dst buffer

not complete until done

completion record

must stay address-valid

task state

Send if spawned

`tokio::spawn` pushes toward 'static + Send; Pin matters after polling.

Project-specific point: hardware may still own pointers while Rust wants to move or drop values. The async API must make that impossible, not merely undocumented.

C++ coroutine: syntax-level suspension

Takeaway: C++ coroutines make async code look sequential, but libraries still provide scheduling and I/O policy.

What the programmer sees

```
submit_descriptor();  
co_await completion;  
continue_with_result();
```

Sequential-looking code, with `co_await` marking a possible suspension point.

What the mechanism stores

locals

saved across suspend

suspend point

where to resume

handle

resume / destroy

Awaiter/promise code decides whether to suspend and how resumption is triggered.

Advisor framing: coroutine syntax is a clean way to express continuation, but it is not by itself an accelerator runtime.

Advisor comparison: what each mechanism makes explicit

Takeaway: choose the mechanism based on which boundary we want to study: graph composition, Rust safety/lifetimes, or syntax-level suspension.

Mechanism	What is explicit?	Who resumes?	Project value / risk
stdexec	Operation graph, receiver contract, scheduler composition.	Scheduler starts operation; receiver receives completion signal.	Good for controlled layer-removal experiments.
Tokio/Rust	Future::poll, Waker, task state, ownership/lifetimes.	Runtime polls again after wakeup.	Best fit for Rust integration; hard part is buffers and completion records across .await.
C++ coroutine	co_await, coroutine frame, awaiter/promise hooks.	Awaiter/runtime resumes coroutine handle.	Good syntax model; policy and runtime still need to be supplied.

Same lifecycle, different proof obligations. For this project, the distinctive Rust work is making lifetime and ownership obligations visible in the API.

Backup: the Rust lifetime challenge in one example

Use this only if the advisor asks why the Tokio path is harder than “just wake a task.”

Pseudo API shape

```
memmove(&src, &mut dst).await  
// hardware is still running
```

The borrow spans the async gap: software is suspended, but hardware may still read or write memory.

What must remain true

1. src cannot be dropped while hardware may read it.
2. dst cannot be treated as complete until completion.
3. Completion record must stay valid and address-stable.
4. Spawned task state must be 'static + Send when it can move across runtime threads.
5. Pin prevents invalid movement of address-sensitive future state.

Rust makes these constraints visible. The design task is to encode them cleanly instead of relying on programmer discipline.